Uniprocessor Scheduling: Mock Test — Solutions

Chapter 9 · 30 marks · Worked solutions

COS 122 Operating Systems

Swift Tuition | 2026

Before you begin

Mark allocations are shown in square brackets.

Rules for every question	A process that arrives at time t is ready for the decision at time t. A lower priority number means a higher priority. Waiting time = turnaround time − service time.
Course	COS 122 Operating Systems, Chapter 9: Uniprocessor Scheduling

Questions

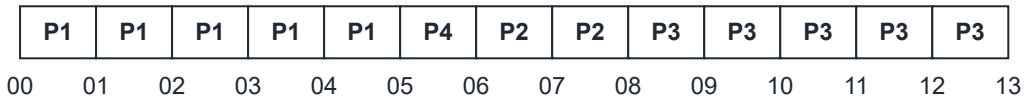
1. Policies from schedules, and round robin	5	p. 3
2. Feedback and HRRN	6	p. 5
3. Shortest remaining time and the measures	7	p. 7
4. Priority scheduling	5	p. 9
5. Types of scheduling	4	p. 11
6. Policy characteristics	3	p. 12
Total	**30**	

Question 1: Policies from schedules, and round robin [5]

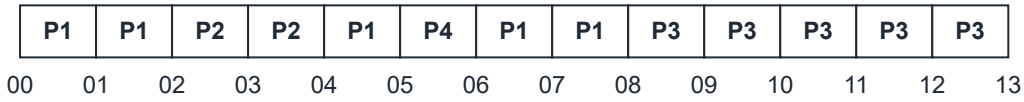
Consider the following workload:

Process	Arrival time	Service time
P1	0	5
P2	2	2
P3	3	5
P4	5	1

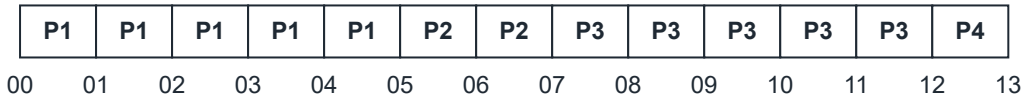
(a) Name a policy that results in the following schedule for the workload given above. **(1)**



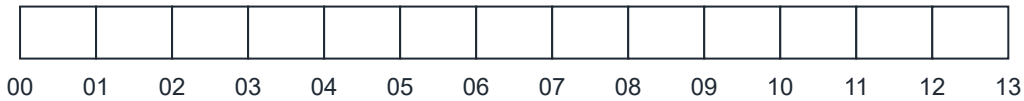
(b) Name a policy that results in the following schedule for the workload given above. **(1)**



(c) Name a policy that results in the following schedule for the workload given above. **(1)**



(d) Show the schedule using the **Round Robin** policy with time quantum **q = 2**. If a quantum ends at the same time as a new process arrives, the new process joins the queue first. Write your solution into the empty boxes of the time scale diagram below. **(2)**



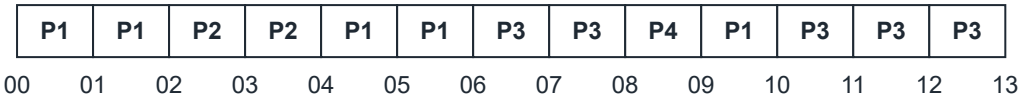
Solution

(a) Shortest Process Next (SPN). No process is interrupted. At time 5 the ready processes are P2 (s = 2), P3 (s = 5) and P4 (s = 1); P4, the shortest, runs, although P2 and P3 arrived earlier. [1 mark for the policy]

(b) Shortest Remaining Time (SRT). P1 is interrupted at time 2, when P2 arrives (P2 needs 2, P1 has 3 left). P4 interrupts P1 at 5 (P4 needs 1, P1 has 2 left). Interruptions happen only at arrivals. [1 mark for the policy]

(c) First Come, First Served (FCFS). Each process runs to completion in arrival order. Other policies also fit: round robin with q ≥ 5, or Feedback with a fixed q ≥ 5, never preempts here, so it gives the same schedule. The expected answer is FCFS. [1 mark for the policy]

(d) Round robin, q = 2:



Time	What happens	Ready queue after the choice (front first)	Runs next	Time left (unfinished processes)
0	Arrives: P1 (joins the back of the queue)	empty	**P1** (starts a new quantum of 2)	P1: 5
2	P1's quantum ends. P2 arrives at the same moment, so P2 joins the queue first and P1 goes behind it.	P1	**P2** (starts a new quantum of 2)	P1: 3, P2: 2
3	Arrives: P3 (joins the back of the queue); P2 is in the middle of its quantum, so it continues	P1, P3	P2 continues	P1: 3, P2: 1, P3: 5
4	P2 finishes	P3	**P1** (starts a new quantum of 2)	P1: 3, P3: 5
5	Arrives: P4 (joins the back of the queue); P1 is in the middle of its quantum, so it continues	P3, P4	P1 continues	P1: 2, P3: 5, P4: 1
6	P1's quantum ends and P1 goes to the back of the queue.	P4, P1	**P3** (starts a new quantum of 2)	P1: 1, P3: 5, P4: 1
8	P3's quantum ends and P3 goes to the back of the queue.	P1, P3	**P4** (starts a new quantum of 2)	P1: 1, P3: 3, P4: 1
9	P4 finishes	P3	**P1** (starts a new quantum of 2)	P1: 1, P3: 3
10	P1 finishes	empty	**P3** (starts a new quantum of 2)	P3: 3
12	P3 uses its whole quantum, but no other process is ready, so P3 keeps running and starts a new quantum	empty	**P3** continues	P3: 1
13	P3 finishes	empty	–	all done

At time 2, P1's quantum ends at the moment P2 arrives: P2 joins the queue first, so P2 runs next. [2 marks: all boxes correct. **Assumed** partial credit: 1 mark if only one or two boxes are wrong.]

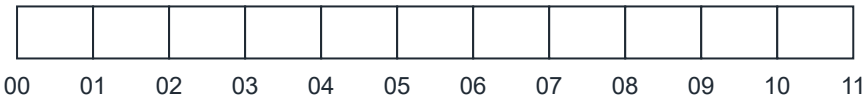
Question 2: Feedback and HRRN [6]

Consider the following workload:

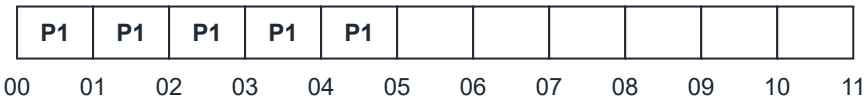
Process	Arrival time	Service time
P1	0	5
P2	1	3
P3	2	2
P4	4	1

(a) Show the schedule using the **Feedback** policy with time quantum $\mathbf{q = 2^i}$. Write your solution into the empty boxes of the time scale diagram below. **(2)**

> **Feedback rules for this question**
>
> - Quantum: $q = 2^i$: RQ0 has q = 1, RQ1 has q = 2, RQ2 has q = 4, RQ3 has q = 8, and so on.
> - A new process joins the back of RQ0 (RQ means ready queue).
> - The scheduler runs the front process of the highest queue that is not empty. Each queue is FCFS.
> - Preemption happens only when a quantum ends. A process that arrives in the middle of a quantum waits.
> - If a process uses its whole quantum and another process is waiting, it moves down one queue. If nothing else is waiting, it stays and starts a new quantum.
> - There is no lowest queue: the number of queues is not limited, so a process that is preempted always moves down one more queue (the textbook rule).



(b) The schedule using the **Highest Response Ratio Next** policy looks as follows after 5 time steps:

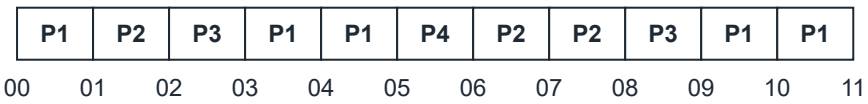


(i) Calculate the response ratio of each ready process at time 5. **(3)**

(ii) Which process will be executed next? **(1)**

Solution

(a) Feedback, $q = 2^i$:



Time	What happens	Ready queue(s) after the choice	Runs next	Time left (unfinished processes)
0	Arrives: P1 (joins RQ0)	all empty	**P1** from RQ0 (q = 1)	P1: 5
1	Arrives: P2 (joins RQ0); P1 uses its whole quantum in RQ0 and moves down to RQ1	RQ1: P1	**P2** from RQ0 (q = 1)	P1: 4, P2: 3
2	Arrives: P3 (joins RQ0); P2 uses its whole quantum in RQ0 and moves down to RQ1	RQ1: P1, P2	**P3** from RQ0 (q = 1)	P1: 4, P2: 2, P3: 2
3	P3 uses its whole quantum in RQ0 and moves down to RQ1	RQ1: P2, P3	**P1** from RQ1 (q = 2)	P1: 4, P2: 2, P3: 1
4	Arrives: P4 (joins RQ0); P1 is in the middle of its quantum; Feedback preempts only when a quantum ends, so P1 continues	RQ0: P4; RQ1: P2, P3	P1 continues	P1: 3, P2: 2, P3: 1, P4: 1
5	P1 uses its whole quantum in RQ1 and moves down to RQ2	RQ1: P2, P3; RQ2: P1	**P4** from RQ0 (q = 1)	P1: 2, P2: 2, P3: 1, P4: 1
6	P4 finishes	RQ1: P3; RQ2: P1	**P2** from RQ1 (q = 2)	P1: 2, P2: 2, P3: 1
8	P2 finishes	RQ2: P1	**P3** from RQ1 (q = 2)	P1: 2, P3: 1
9	P3 finishes	all empty	**P1** from RQ2 (q = 4)	P1: 2
11	P1 finishes	all empty	–	all done

[2 marks: all boxes correct. **Assumed** partial credit: 1 mark if only one or two boxes are wrong.]

(b) Response ratio R = (w + s)/s, where w = time waited so far and s = service time.

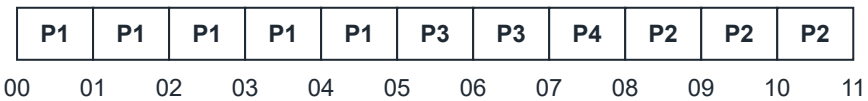
(i)

Process	w	s	R
P2	5 − 1 = 4	3	(4 + 3)/3 = **2.33**
P3	5 − 2 = 3	2	(3 + 2)/2 = **2.50**
P4	5 − 4 = 1	1	(1 + 1)/1 = **2**

[1 mark per ratio]

(ii) P3, the highest ratio. (SPN would pick P4, the shortest; HRRN also counts the waiting time.) [1 mark]

For reference, the complete HRRN schedule:

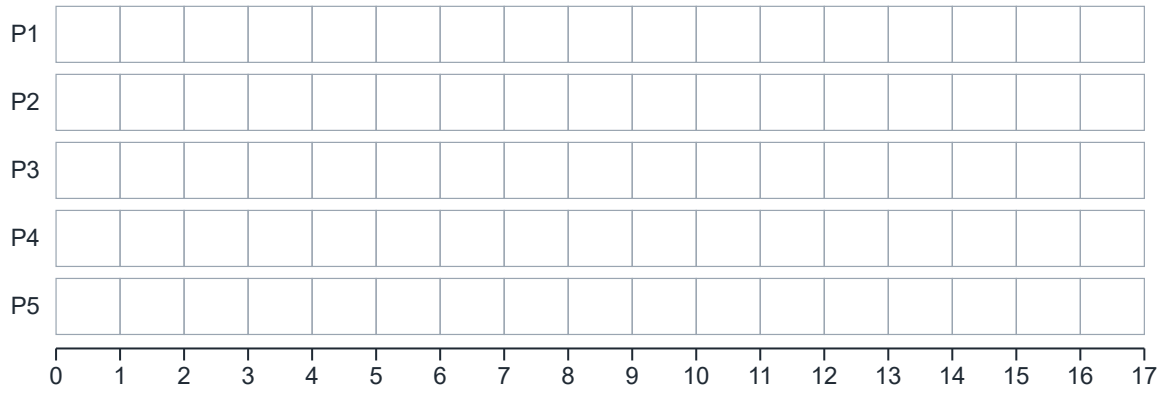


Question 3: Shortest remaining time and the measures [7]

Consider the following workload:

Process	Arrival time	Service time
P1	0	7
P2	2	4
P3	3	1
P4	5	3
P5	6	2

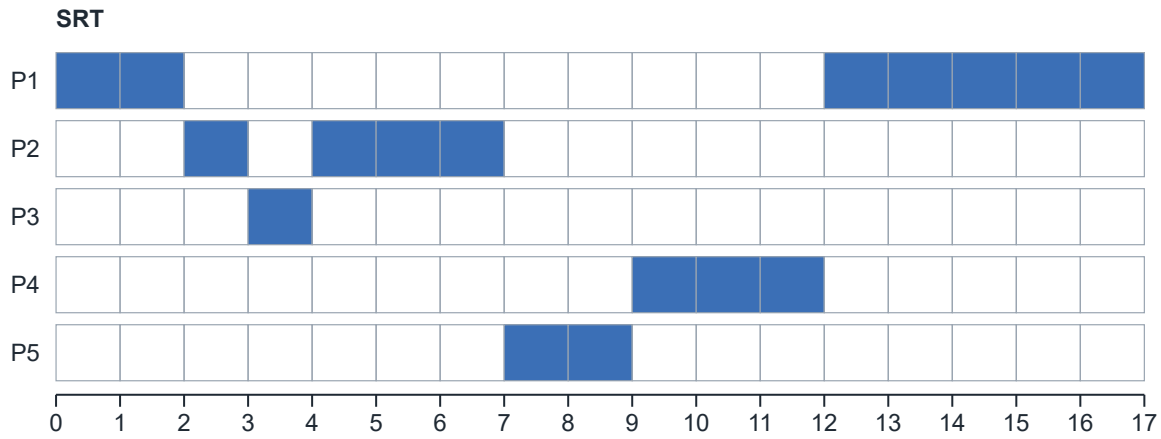
(a) Draw the schedule using the **Shortest Remaining Time (SRT)** policy on the time-scale grid below. A newly arrived process preempts the running process only if it needs strictly less time than the running process has left. **(3)**



(b) Complete the table for your schedule: finish time, turnaround time T_r, normalised turnaround T_r/T_s and waiting time, and the mean of the last three columns. **(4)**

Solution

(a)



Time	Ready processes (time left)	Choice	Reason
0	P1: 7 left	**P1**	P1 is the only process ready
2	P1: 5 left (running), P2: 4 left	**P2**	P2 needs 4, less than the 5 that P1 still needs, so P2 preempts P1
3	P1: 5 left, P2: 3 left (running), P3: 1 left	**P3**	P3 needs 1, less than the 3 that P2 still needs, so P3 preempts P2
4	P1: 5 left, P2: 3 left	**P2**	P2 has the least time left (3)
5	P1: 5 left, P2: 2 left (running), P4: 3 left	**P2**	P4 arrives needing 3; P2 has 2 left. 3 is not less than 2, so P2 keeps running
6	P1: 5 left, P2: 1 left (running), P4: 3 left, P5: 2 left	**P2**	P5 arrives needing 2; P2 has 1 left. 2 is not less than 1, so P2 keeps running
7	P1: 5 left, P4: 3 left, P5: 2 left	**P5**	P5 has the least time left (2)
9	P1: 5 left, P4: 3 left	**P4**	P4 has the least time left (3)
12	P1: 5 left	**P1**	P1 is the only process ready

[3 marks: 1 for each correct preemption decision (times 2 and 3) and 1 for the rest of the schedule. **Assumed** split.]

(b)

Process	Arrival	Service T_s	Finish	Turnaround T_r = finish − arrival	T_r/T_s	Waiting = T_r − T_s
P1	0	7	17	17 − 0 = 17	17/7 = 2.43	17 − 7 = 10
P2	2	4	7	7 − 2 = 5	5/4 = 1.25	5 − 4 = 1
P3	3	1	4	4 − 3 = 1	1/1 = 1	1 − 1 = 0
P4	5	3	12	12 − 5 = 7	7/3 = 2.33	7 − 3 = 4
P5	6	2	9	9 − 6 = 3	3/2 = 1.50	3 − 2 = 1
Mean				**33/5 = 6.60**	**1.70**	**16/5 = 3.20**

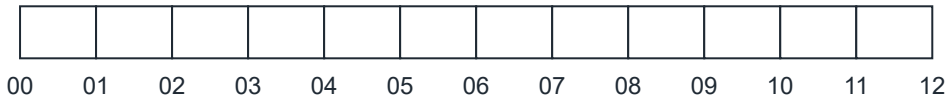
[4 marks: finish times 1, T_r column and mean 1, T_r/T_s column and mean 1, waiting column and mean 1. Follow through from a wrong schedule. **Assumed** split.]

Question 4: Priority scheduling [5]

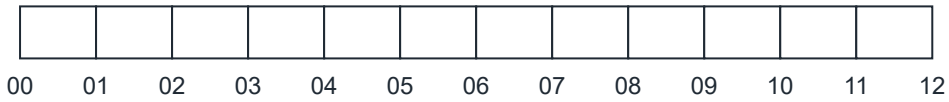
Consider the following workload. A lower priority number means a higher priority.

Process	Arrival time	Service time	Priority
P1	0	4	3
P2	1	3	2
P3	3	2	1
P4	4	3	4

(a) Show the schedule using **non-preemptive priority** scheduling. Write your solution into the empty boxes. **(2)**



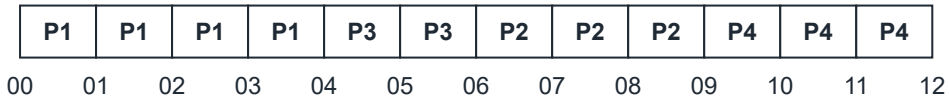
(b) Show the schedule using **preemptive priority** scheduling. A new process preempts the running process only if its priority is strictly higher. Write your solution into the empty boxes. **(2)**



(c) Which process in this workload is at risk of starvation under priority scheduling if more high-priority processes keep arriving, and how can an operating system prevent it? **(1)**

Solution

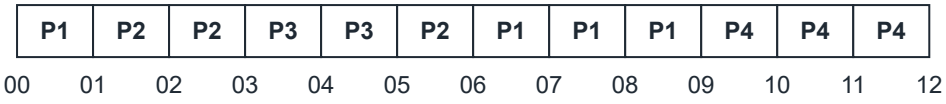
(a)



Time	Ready processes (time left)	Choice	Reason
0	P1: 4 left	**P1**	P1 is the only process ready
4	P2: 3 left, P3: 2 left, P4: 3 left	**P3**	P3 has the highest priority (lowest number, 1)
6	P2: 3 left, P4: 3 left	**P2**	P2 has the highest priority (lowest number, 2)
9	P4: 3 left	**P4**	P4 is the only process ready

[2 marks; **Assumed** partial credit: 1 mark if one or two boxes are wrong.]

(b)



Time	Ready processes (time left)	Choice	Reason
0	P1: 4 left	**P1**	P1 is the only process ready
1	P1: 3 left (running), P2: 3 left	**P2**	P2 has priority 2, higher than P1 (priority 3), so P2 preempts P1
3	P1: 3 left, P2: 1 left (running), P3: 2 left	**P3**	P3 has priority 1, higher than P2 (priority 2), so P3 preempts P2
4	P1: 3 left, P2: 1 left, P3: 1 left (running), P4: 3 left	**P3**	P4 arrives with priority 4; P3 has priority 1. That is not higher, so P3 keeps running
5	P1: 3 left, P2: 1 left, P4: 3 left	**P2**	P2 has the highest priority (lowest number, 2)
6	P1: 3 left, P4: 3 left	**P1**	P1 has the highest priority (lowest number, 3)
9	P4: 3 left	**P4**	P4 is the only process ready

[2 marks; **Assumed** partial credit as in (a).]

(c) P4, the lowest priority (number 4): it runs only when no other process is ready. Prevent starvation with a dynamic priority that rises the longer a process waits (ageing). [1 mark: process and remedy]

Question 5: Types of scheduling [4]

In your own words describe the four types of scheduling. For each type, name the decision it makes. **(4)**

Solution

Type	Description	Decision (1 mark each, description and decision both needed)
Long-term	Admits new jobs into the system; it runs least often.	Whether to add a new program to the pool of processes to be executed; it controls the degree of multiprogramming.
Medium-term	Moves processes between main memory and disk (swapping).	Whether to add to the number of processes that are partly or fully in main memory; part of swapping.
Short-term (dispatcher)	Gives the processor to a ready process; it runs most often, at every interrupt or call that can change the running process.	Which available (ready) process the processor executes next.
I/O	Manages the queue of requests waiting for each I/O device.	Which process's waiting I/O request is handled by an available I/O device.

Question 6: Policy characteristics [3]

(a) Give the selection function and the decision mode (non-preemptive, or preemptive and when) of **SRT** and of **HRRN**. Use w for the time spent waiting so far, e for the time spent executing so far and s for the total service time. **(2)**

(b) Feedback can use the same quantum q = 1 in every queue, or q = 2^i in queue RQi. Give one difference in how these two versions treat a long process. **(1)**

Solution

(a)

Policy	Selection function	Decision mode
SRT	min[s − e]	Preemptive (at arrival)
HRRN	max[(w + s)/s]	Non-preemptive

[0.5 mark per cell]

(b) With q = 1 a long process moves down to a low queue. There it gets only one time unit each time it runs. So its turnaround keeps taking longer, and it can starve. With q = 2^i the lower queues have longer quanta (1, 2, 4, ...). So a long process gets a longer run each time it is chosen, and it is less likely to starve. [1 mark]